Milestones 1.1 and 1.2

Healthcare Appointment Booking System - Supporting Documents

Milestone 1 — Delivery Plan & Status

Milestone 1 ("the reliability spine") is split into three sub-milestones. The day ranges map to the build guide's 2-week sequence.

Sub-milestone	Days	Scope
M1.1 — Foundation & Data Layer	1–3	Schema + Prisma migrations, availability templates, materialized slots, state machine, DB-level constraints
M1.2 — Booking Core & Concurrency	4–7	Row-level locking (`SELECT … FOR UPDATE`), reservation expiry worker + sweep, core booking flow end-to-end, admin CRUD
M1.3 — Payments, Verification & Delivery	8–14	Pix + card integration, signature-verified idempotent webhooks, expiry/payment race handling, concurrency + webhook test suites, deployment (Railway/Render), docs + Postman collection, final hardening

The four client acceptance criteria (no double booking, reliable expiry release, end-to-end Pix + card in sandbox, reproducible setup) are proven in M1.3 but enabled by the constraints and state machine laid down in M1.1.

Environment note (verification on this machine)

This dev machine has no Node/Docker preinstalled and its `D:` drive blocks native-binary writes (Defender). Verification therefore runs from a `C:` mirror with a portable Node 22 and a portable PostgreSQL 16 (no admin). One hard limit: **Prisma's Rust query engine crashes on this CPU** (`0xC000001D` illegal instruction / `0xC0000409` stack overrun, on both `library` and `binary` engines). That means the Prisma-based server and the Prisma integration tests (`test:integration`) cannot run *here* — they run in CI / on Railway/Render, where the Linux engine is fine.

To prove the runtime guarantees on this box regardless, the two critical behaviours are demonstrated with the pure-JS `pg` driver running the **identical SQL** the app uses:

Proof	Command	Result
Concurrency (criterion #1)	`npm run verify:locking`	✅ 50 contenders × 20 rounds — every round exactly 1 success, 49 clean losses, 0 errors
Expiry release (criterion #3)	`npm run verify:expiry`	✅ expired→freed, not-due→untouched, paid→never released

---

M1.1 — Foundation & Data Layer — status: ✅ VERIFIED

Delivered in this sub-milestone:

- [x] Project scaffold (TypeScript, ESM, npm scripts, Docker Postgres, env template).
- [x] Prisma schema: `Doctor`, `AvailabilityTemplate`, `AppointmentSlot`, `Reservation`, `Payment`, `WebhookEvent`, `EventLog`, plus the three status enums.
- [x] `init` migration (all tables, indexes, FKs) authored to match the schema.
- [x] `hard_constraints` migration — the guarantees Prisma can't express:
  - `one_live_slot_hold` partial unique index (a slot holds ≤ 1 live claim).
  - `no_overlap_per_doctor` GiST exclusion constraint (needs `btree_gist`) — a doctor can never have two overlapping slots.
- [x] Config plumbing: zod-validated env loader, pino logger (with secret redaction), Prisma client singleton, transactional `logEvent` helper.
- [x] Reservation/slot **state machine** encoded as data (`canTransition` / `assertTransition`) + full unit-test coverage of legal & illegal moves.
- [x] Availability templates → **slot materialization**: pure slot math (`iterateSlots`) + idempotent DB upsert (`materializeSlots`) over a rolling `SLOT_MATERIALIZATION_WEEKS` window, with limited-Sunday-hours handling.
- [x] Unit tests for slot math (spacing, custom slot length, trailing-partial drop, narrow Sunday window, empty/inverted windows).

- [x] Idempotent seed (demo doctor + weekday/Saturday/Sunday templates + materialized slots).

Verified: unit tests (12/12), full `tsc --noEmit`, migrations applied to a real Postgres, and the two hard constraints **observed rejecting bad data** (overlapping slot → rejected by `no_overlap_per_doctor`; duplicate `(doctorId, startsAt)` → rejected by the unique index; non-overlapping → accepted).

M1.2 — Booking Core & Concurrency — status: ✅ code complete, core guarantees proven

Delivered:

- [x] `reserveSlot` — raw `SELECT … FOR UPDATE` inside a Prisma interactive transaction, with lock-wait tx options tuned for concurrent bursts.
- [x] **Concurrency proven**: 50 × 20 rounds, exactly one winner every round (`npm run verify:locking`).
- [x] Expiry: `releaseReservation` (locked re-check), periodic `sweep`, precise pg-boss `scheduleExpiry`, and the `worker.ts` entrypoint wiring all three plus nightly materialization. **Release logic proven** across all three cases (`npm run verify:expiry`).
- [x] Reservation service (`reserve` + best-effort schedule; sweep is backstop).
- [x] Patient booking API — `GET /api/doctors`, `GET /api/slots`, `POST /api/reservations`, `GET /api/reservations/:id`, payment-start stub (501 until M1.3), plus `/healthz` + `/readyz`.
- [x] Admin CRUD behind an API-key guard — doctors, availability templates, on-demand materialize.
- [x] Central error handling (409 for lost races, 400 for bad input, never 500), zod validation, async handler wrapper.
- [x] Prisma integration tests authored (`tests/integration/reserve-race`, `tests/integration/expiry`) for CI/deploy where the Prisma engine runs.
- [x] Whole M1.2 tree typechecks clean (`tsc --noEmit` exit 0).

Not runnable on this machine: the Express server and `test:integration` (both need the Prisma runtime — see the environment note above). They are typechecked here and run in CI / deployment.

M1.3 — Payments, Verification & Delivery — status: 🟡 payment layer complete + proven; finalization pending

Provider chosen: **AbacatePay** (Pix-first). Built provider-agnostic so only the concrete adapter changes when sandbox keys arrive.

Delivered + proven:

- [x] Swappable `PaymentProvider` interface + AbacatePay adapter skeleton (endpoints/fields/signature scheme marked `TODO(sandbox)`) + in-memory `MockProvider` + provider registry.
- [x] HMAC-SHA256 raw-body signature verification (constant-time compare).
- [x] `startPayment` — stable idempotency key per (reservation, method); one Payment row; provider-idempotent charge.
- [x] Signature-verified, idempotent **webhook handler** with the full exception matrix, all under the same row lock the expiry job uses.
- [x] Raw-body webhook route mounted before `express.json()`; real payment-start route; 401 on tampered signatures; 409 on unpayable reservations.
- [x] **Proven on this box** (`npm run verify:webhook`): idempotency (5× sequential + 5× concurrent → confirm once, charge once), out-of-order convergence, tampered-signature rejection, and the **payment-at-expiry race consistent across 50 rounds** (always Paid+Approved OR freed+Refunded — never paid-but-unbooked).
- [x] Prisma integration tests authored (`tests/integration/webhook`) for CI.
- [x] Whole tree + tests typecheck clean (`tsc --noEmit`, `tsc -p tsconfig.test.json`).

Finalization still to do (the PDF's close-out phase):

- [ ] Wire the AbacatePay adapter against the real sandbox (needs API keys) and run one live Pix + one live card end-to-end with captured evidence.
- [ ] Security hardening: `helmet`, `express-rate-limit` on reserve/payment, HTTPS-only in prod, DB least-privilege role.
- [ ] k6 load suite in CI + docs package (`SCHEMA.md`, `API.md`, `WEBHOOKS.md`, `DEPLOYMENT.md`) + Postman collection.
- [ ] Deploy web + worker to Railway/Render; reproducibility check on a clean machine; client acceptance walkthrough + written sign-off.

Booking Core — Local Setup

Healthcare appointment booking system (Milestone 1: the reliability spine). Stack: **Node.js · Express · PostgreSQL · Prisma · pg-boss**, deployed on Railway/Render.

> **Status:** Milestone 1.1 (foundation & data layer) is code-complete. The booking API, worker, and payments arrive in M1.2 / M1.3 — see `../MILESTONES.md`.

Prerequisites

- Node.js LTS (v20 or v22)
- Docker (for local Postgres) — or any PostgreSQL 15+ instance
- npm

One-time setup

```
# 1. Start Postgres (local Docker)
docker compose up -d

# 2. Configure environment
cp .env.example .env        # then edit values as needed

# 3. Install dependencies
npm install

# 4. Generate the Prisma client + apply migrations
npx prisma generate
npx prisma migrate deploy   # applies init + hard_constraints (needs btree_gist)

# 5. Seed a demo doctor, templates, and materialized slots
npm run seed
```

Everyday commands

Command	What it does
`npm test`	Run the unit test suite (state machine + slot math)
`npm run test:integration`	Prisma-based integration tests (needs a running DB)
`npm run verify:locking`	Prove the no-double-booking guarantee (50×20 concurrency rounds)
`npm run verify:expiry`	Prove expired reservations release and paid ones never do
`npm run seed`	(Re)seed the demo clinic and materialize slots
`npx prisma studio`	Browse the database in a GUI
`npm run build`	Compile TypeScript to `dist/`
`npm run dev`	Run the web server (API) with hot reload
`npm run worker`	Run the background worker (expiry + sweep + materialization)

What exists today (M1.1)

- **Schema & migrations** — all tables, indexes, and the two hard DB constraints (`one_live_slot_hold`, `no_overlap_per_doctor`).
- **State machine** — `src/domain/stateMachine.ts`, the pure safety net for all slot transitions.

- **Slot materialization** — `src/slots/` turns availability templates into concrete, lockable `AppointmentSlot` rows.
- **Plumbing** — env validation, logging, Prisma client, event logging.

Verifying the constraints (after `migrate deploy`)

```
-- The GiST exclusion constraint should reject an overlapping slot for a doctor:
\d "AppointmentSlot"      -- shows no_overlap_per_doctor
-- The partial unique index guarding live holds:
\di one_live_slot_hold
```

A full `SCHEMA.md`, `API.md`, `WEBHOOKS.md`, `DEPLOYMENT.md`, and the Postman collection are M1.3 deliverables.

Data Model & Constraint Reasoning

The database is the source of truth. Every safety guarantee is enforced by a DB constraint **and** application logic — never app logic alone.

Tables

Table	Purpose
`Doctor`	A practitioner; owns availability templates and slots.
`AvailabilityTemplate`	Weekly recurring availability (day-of-week, start/end, slot length). Limited Sunday hours are just a narrower window.
`AppointmentSlot`	A concrete, bookable time slot (materialized from templates). The row that gets locked during booking.
`Reservation`	A patient's hold on a slot, with a TTL (`reservedUntil`).
`Payment`	A charge attempt against a reservation, with a stable idempotency key.
`WebhookEvent`	Ledger of received provider events — the storage-layer idempotency guard.
`EventLog`	Append-only log of every state transition (audit trail).

Status enums

- **SlotStatus**: `Available → Reserved → Paid`, plus `Expired`, `Cancelled`.

- **ReservationStatus**: `Active → Paid` / `Expired` / `Cancelled`.
- **PaymentStatus**: `Pending → Approved` / `Rejected` / `Refunded`.

Legal transitions are enforced in code by the state machine (`src/domain/stateMachine.ts`).

Constraints and why they exist

Constraint	Table	Why
`@@unique([doctorId, startsAt])`	AppointmentSlot	A doctor can't have two slots at the same instant; also makes slot **materialization idempotent** (re-runs can't duplicate).
`no_overlap_per_doctor` (GiST exclusion, `btree_gist`)	AppointmentSlot	A doctor can never have two **overlapping** slots — enforced by the DB, not app logic.
`Reservation.slotId @unique`	Reservation	At most one reservation per slot — the core anti-double-booking guard.
`one_live_slot_hold` (partial unique index)	AppointmentSlot	Documents/guards that a slot holds at most one live (`Reserved` / `Paid`) claim.
`Payment.idempotencyKey @unique`	Payment	A retried "start payment" can't create a second charge.
`@@unique([provider, providerEventId])`	WebhookEvent	Duplicate/retried webhooks are a no-op — confirm once, charge once.

Concurrency model

Booking uses a raw `SELECT … FOR UPDATE` inside a Prisma interactive transaction: concurrent callers block on the slot row, then read the updated status and lose cleanly. The expiry job and the payment webhook take the **same** row lock, which is what makes the payment-at-expiry race resolve to exactly one consistent outcome. See `WEBHOOKS.md` for the exception matrix.

API Reference

Base URL: the deployed web service. All bodies are JSON. Errors return a JSON `{ "error": "...", "message"?: "..." }` with an appropriate status — never a bare 500 for expected conditions.

Health

Method	Path	Notes
GET	`/healthz`	Liveness. `{ "status": "ok" }`
GET	`/readyz`	Readiness incl. a DB round-trip.

Patient booking

`GET /api/doctors`

List active doctors. → `{ "doctors": [{ "id", "name", "specialty" }] }`

`GET /api/slots?doctorId=<uuid>&date=<YYYY-MM-DD>`

Available slots for a doctor on a date (UTC). → `{ "slots": [{ "id", "startsAt", "endsAt", "status" }] }` Errors: `400` if `doctorId` / `date` malformed.

`POST /api/reservations`

Reserve a slot (row-locked). Rate-limited. Body: `{ "slotId", "patientName", "patientEmail", "patientPhone" }` → `201 { "id", "slotId", "status": "Active", "reservedUntil" }` Errors: `409 slot_unavailable` if already taken; `400` on invalid body; `429 rate_limited`.

`POST /api/reservations/:id/payment`

Start a Pix or card payment for an active reservation. Rate-limited. Body: `{ "method": "pix" | "card" }` → `201 { "paymentId", "providerRef", "pixQr"?, "checkoutUrl"?, "status": "Pending" }` Errors: `409 reservation_not_payable`; `400` on invalid body.

`GET /api/reservations/:id`

Status polling for the UI. → `{ "reservation": { "id", "status", "reservedUntil", "slot": { "id", "status", "startsAt", "endsAt" } } }` Errors: `404 reservation_not_found`.

Payment webhook

`POST /webhooks/payment`

Provider callback. Raw body; HMAC-signature verified. Idempotent. → `200 { "status": "processed" }` | `200 { "status": "duplicate_ignored" }` | `401 invalid_signature` | `400`. See `WEBHOOKS.md` for the exception matrix.

Admin (guarded by `x-admin-api-key`)

Method	Path	Body / Notes
POST	`/admin/doctors`	`{ "name", "specialty"? }`
GET	`/admin/doctors`	list all
PATCH	`/admin/doctors/:id`	`{ "name"?, "specialty"?, "active"? }`
POST	`/admin/doctors/:id/availability`	`{ "dayOfWeek" 0-6, "startTime" "HH:MM", "endTime" "HH:MM", "slotMinutes"?, "active"? }`
POST	`/admin/materialize`	Generate concrete slots now. → `{ "created": <n> }`

All admin routes return `401 unauthorized` without a valid `x-admin-api-key`.

Payment Webhooks

Endpoint: `POST /webhooks/payment` (provider: **AbacatePay**).

Signature verification

- The route parses the **raw request body** (`express.raw`) — mounted before the global JSON parser — because signatures are computed over the exact bytes.
- Verification is **HMAC-SHA256** of the raw body keyed with `PAYMENT_WEBHOOK_SECRET`, compared in constant time. A `sha256=` prefix on the header is tolerated.
- A missing or invalid signature returns **401** and makes no state change.

> The exact AbacatePay signature header/scheme is confirmed against their sandbox docs and adjusted only inside `src/payments/abacatepay.ts` (`verifyWebhook`). Everything downstream is provider-agnostic.

Idempotency

Every event is recorded in `WebhookEvent` with a unique `(provider, providerEventId)` index. A replayed or retried delivery hits that unique constraint and returns **200** `duplicate_ignored` without reprocessing — so duplicate/retried webhooks **confirm once and charge once**. Decisions key off payment **status**, not arrival order, so out-of-order delivery is safe.

Exception matrix

All branches run inside one transaction that takes `SELECT … FOR UPDATE` on the reservation **and** its slot — the same lock the expiry job uses — so the payment-at-expiry race always resolves to exactly one consistent outcome.

Event	Slot/reservation state	Action	Result
`approved`	slot `Reserved`, reservation `Active`	slot → `Paid`, reservation → `Paid`, payment → `Approved`	Booking confirmed
`approved`	slot already released / re-sold / expired	payment → `Refunded`, dispatch provider refund	Charge voided, **never** a phantom booking
`rejected`	any	payment → `Rejected`	Slot left to expire via its TTL
`pending`	any	no-op (logged)	Awaiting a terminal event
any (replay)	duplicate `providerEventId`	none	`200 duplicate_ignored`
invalid signature	—	none	`401`

Payment-at-expiry outcomes (both correct)

Outcome	Meaning
Slot `Paid`, reservation `Paid`, payment `Approved`	Webhook won the row lock first
Slot freed (`Available` / `Expired`), payment `Refunded`	Expiry won; charge auto-voided

The forbidden state — payment `Approved` but slot not `Paid` (charged with no booking) — is impossible because the confirm branch only fires while the slot is still `Reserved` under the lock. Proven across 50 concurrent rounds by `npm run verify:webhook`.

Refund dispatch

The refund (provider I/O) is dispatched **after** the transaction commits, so no row lock is held across a network call. If the refund call fails the payment is already marked `Refunded` and the failure is logged for manual/ops retry.

> Production hardening (noted, not yet built): move refund dispatch to a durable pg-boss job with automatic retries.

Deployment Runbook (Railway / Render)

The system runs as **two processes from one repo/image**, sharing one managed PostgreSQL:

Service	Command	Role
web	`npx prisma migrate deploy && node dist/server.js`	HTTP API + webhook receiver
worker	`node dist/worker.js`	Expiry scheduling, sweep, nightly materialization

pg-boss stores its queue in the same Postgres, so no Redis is needed for M1.

Railway (primary)

1. **Provision Postgres** — New → Database → PostgreSQL. Copy its `DATABASE_URL` (the `.internal` URL for service-to-service).
2. **Web service** — New → GitHub repo → this repo. Railway reads `railway.json` (Dockerfile build, start command with migrate-on-release, `/healthz` healthcheck).
3. **Worker service** — New → same repo → set **start command** to `node dist/worker.js` (and skip the healthcheck).
4. **Environment variables** (both services) — from `.env.example`:
   - `DATABASE_URL` (reference the Postgres service)
   - `PAYMENT_PROVIDER=abacatepay`, `PAYMENT_API_KEY`, `PAYMENT_WEBHOOK_SECRET`, `PAYMENT_API_BASE_URL`
   - `ADMIN_API_KEY` (a strong secret)
   - `RESERVATION_TTL_MINUTES`, `SLOT_MATERIALIZATION_WEEKS`, `DEFAULT_PRICE_CENTS`
   - `TRUST_PROXY=true`, `FORCE_HTTPS=true`, `NODE_ENV=production`
5. **btree_gist** — created automatically by the `hard_constraints` migration during `prisma migrate deploy`; no manual step.
6. **Register the webhook** — in the AbacatePay dashboard, point the payment webhook at `https://<web-domain>/webhooks/payment`.
7. **Smoke test** — run one sandbox Pix and one card payment against the live URL end-to-end; confirm a reservation expires in prod (worker is running).

Render (alternative)

- **Web Service** — Docker; start `npx prisma migrate deploy && node dist/server.js`; health check `/healthz`.
- **Background Worker** — same image; start `node dist/worker.js`.
- **PostgreSQL** — managed instance; wire `DATABASE_URL` into both.

- Same env vars as above.

Release checklist

- [ ] `DATABASE_URL` points at the managed Postgres (private URL).
- [ ] All env vars set on **both** services; secrets are real sandbox keys.
- [ ] Web deploy shows migrations applied (`migrate deploy` in the logs).
- [ ] Worker deploy is running (a test reservation expires after its TTL).
- [ ] Webhook URL registered and reachable; a test event returns 200.
- [ ] `TRUST_PROXY=true` and `FORCE_HTTPS=true` in production.
- [ ] One live Pix and one live card payment completed end-to-end.

Rollback

Redeploy the previous image/commit. Migrations are additive in M1; no destructive down-migrations are used.

Milestone 1 — Acceptance Evidence

A living record of the proof behind each acceptance criterion. Everything here is reproducible from the repo; CI re-proves it on every push.

Automated CI (Linux + PostgreSQL)

- **CI workflow:** `.github/workflows/ci.yml` — install → prisma generate → **migrate deploy** → typecheck → unit tests → **integration suite (reserve-race, expiry, webhook)** → concurrency/expiry/webhook proofs.
  - green run: https://github.com/KM99999/my-client/actions/runs/28630825510
- **Load workflow:** `.github/workflows/load.yml` — boots the real Express + Prisma server on Linux, runs k6 (50 VUs) at one contended slot, then asserts **exactly one reservation won**.
  - green run: https://github.com/KM99999/my-client/actions/runs/28630825547

The integration suite runs the guarantees through the **real application code** (Prisma `reserveSlot`, `releaseReservation`, the webhook handler) on Linux — not just the standalone scripts.

Criterion → evidence

#	Client criterion	Evidence
1	No double booking under load	`verify:locking` — 50 concurrent × 20 rounds, exactly one winner each round; integration `reserve-race`; k6 HTTP load test (`load.yml`) asserting exactly one reservation.
2	Duplicate/retried webhooks confirm once, charge once	`verify:webhook` — same event ×5 (sequential + concurrent) → one confirmation, one charge; out-of-order convergence; integration `webhook`.
3	Expired reservations auto-release	`verify:expiry` + integration `expiry` — expired→freed, not-due→held, paid→never freed; precise job **and** sweep paths.
4	Payment-at-expiry always consistent	`verify:webhook` race — 50 rounds always resolve to Paid+Approved **or** freed+Refunded, never paid-but-unbooked.
—	Reproducible setup	`prisma migrate deploy` from a clean DB (CI does this every run); `docs/RUN_ONLINE.md` + `docs/README.md`.

Reproduce locally

```
npm test                 # unit
npm run test:integration # Prisma integration (needs a DB)
npm run verify:locking   # concurrency proof
npm run verify:expiry    # expiry proof
npm run verify:webhook   # idempotency + payment-at-expiry proof
```

Pending (external dependencies)

- Live Pix + card sandbox runs (needs AbacatePay keys) — recordings to be added here.
- Staging deployment URL (needs Railway access) — to be added here.

Run it in a browser (GitHub Codespaces or Gitpod)

Milestone 1.1 is the data + logic foundation (no screen of its own). **Milestone 1.2 adds the booking screen** — a reference widget that walks a patient through doctor → date/time → details → a live held slot with a countdown. That widget is the thing to show the client.

Replace `<OWNER>/<REPO>` below with your repository once it's pushed.

Option A — GitHub Codespaces (recommended)

1. Push this repo to GitHub (see the repo's push instructions).
2. Open `https://codespaces.new/<OWNER>/<REPO>` — or on the repo page: **Code** ▸ **Codespaces** ▸ **Create codespace on main**.
3. Wait for the container to build. The `.devcontainer` automatically: installs deps, generates the Prisma client, **applies the migrations**, and **seeds** a demo clinic. (First build takes a couple of minutes.)

Option B — Gitpod

Open `https://gitpod.io/#https://github.com/<OWNER>/<REPO>`. The `.gitpod.yml` runs install ▸ generate ▸ migrate ▸ seed, then `npm test`.

See the booking screen (the client-facing visual)

In a Codespace / Gitpod (uses the real Express + Prisma app):

```
npm run dev
```

Then open the forwarded **port 3000** (Codespaces pops a "Open in Browser" prompt). You'll see the booking widget backed by the real API.

On any machine with Postgres but no Prisma (e.g. the original Windows box):

```
npm run preview      # pg-backed demo server, seeds a doctor + 14 days of slots
# open http://localhost:3000
```

Either way you get: pick a doctor → pick a date & time → enter details → the slot is **held with a live countdown** (the M1.2 reservation TTL), and the payment button shows that Pix + card land in Milestone 1.3.

Prove the guarantees in the terminal (Milestone 1.1)

Once the environment is ready, run these in the built-in terminal:

```
# 1. Unit tests — state machine + slot materialization math (12 tests)
npm test

# 2. Full typecheck
npm run typecheck
```

```
# 3. DB-level constraint proof (overlap + duplicate rejected, valid accepted)
psql "$DATABASE_URL" -f scripts/verify-m11-constraints.sql

# 4. The concurrency + expiry guarantees, live against Postgres
npm run verify:locking     # 50 concurrent bookings on one slot -> exactly 1 wins
npm run verify:expiry      # expired -> freed, not-due -> held, paid -> never freed
```

Everything above is Milestone 1.1 (plus the M1.2 concurrency/expiry guarantees it enables). To browse the data visually:

```
npx prisma studio          # opens a DB GUI (forwarded to your browser)
```

What you should see

- `npm test` → **12 passed**.
- The constraint proof → four `PASS` lines (overlap rejected, duplicate rejected, adjacent slot accepted, final count = 2).
- `verify:locking` → 20 rounds, each **exactly 1 success / 49 clean losses**.

> Note: unlike this project's original Windows dev box, the cloud Linux runtime runs Prisma's engine, so the Express API (`npm run dev`) and the Prisma integration tests (`npm run test:integration`) also work here.